Test og rapportering

Opgavebeskrivelse

Fortsættelse af sidste uge: Test din kollegas PDF-downloader

Din leder og dig blev enige om at kvalitetssikre den foreslåede løsning til download af PDF-rapporter. Løsningen er blevet udviklet af en kollega, men vedkommende har oplevet ustabilitet og performanceproblemer, og vil derfor gerne have sikret dens funktionalitet gennem en grundig teststrategi.

I samråd med din leder kommer I frem til følgende tilgang:

- Enhedstest af kritiske funktioner i PDF-downloaderen.
- Integrationstest for at sikre samspillet mellem Excel-dataindlæsning, URL-håndtering og filhåndtering.
- Robust fejlhåndteringstest, hvor systemet valideres mod forskellige fejlscenarier (f.eks. ugyldige links, timeout, netværksfejl).
- En tilbagerapportering til den oprindelige udvikler af identificerede fejl og forbedringer baseret på din analyse og arbejde med kodebasen.

Du får et link til en af dine kollegaers repository. Opret en fork heraf hvori du tilføjer tests og dine ændringer af programmet.

OBS: For ikke at overbelaste Specialisternes netværk - Hvis I beslutter jer for at teste PDF-downloaderens concurrency, så sørg for at teste med maks 10 PDF'er ad gangen, når I er på kontoret.

Github og præsentation

- Opgavens løsning skal uploades i et github-repository der indeholder følgende:
    - Enhedstest (Unit Tests) af kritiske funktioner i PDF-downloaderen fra sidste uge
    - Integrationstest mellem PDF-downloaderen og Excel/CSV-filhåndtering
    - Testrapport, der viser testdækningen, forklarer hvordan fejl og mangler blev håndteret og som reflekterer over kodens kvalitet samt forbedringsforslag.

- README-fil med beskrivelse af hvordan man kører test suiten

- Forbered en 10-15 minutters præsentation til på fredag af din PDF-downloader fra første uge. Vi anbefaler at fokusere på følgende elementer:
  - Forklar din tilgang til opgaven og kundens forespørgsel
  - Demo af din pdf-downloader
  - Nedslag i kodebasen
    - Hvordan er din løsning struktureret?
    - Download-logik, håndtering af filer og URL'er
  - Forsøg at inddrage feedback og ændringer på baggrund af din kollegas arbejde med din kodebase
    - Hvad viste testarbejdet?
    - Hvilke områder er blevet forbedret og hvordan?
    - NB: Det kræver at du snakker sammen med din kollega i løbet af torsdag og fredag formiddag
  - Reflekter over arbejdet med PDF-downloaderen og oplevelsen af at bytte kodebase

---

Foreslået fremgangsmåde:

Del 1: Implementering af enhedstest (Unit Tests)

- Skriv unit tests for hver af PDF-downloaderens kernefunktioner.
  - Hvis der er unittests i forvejen, så vurder hvorvidt de kan forbedres
- Anvend mocking til at isolere testene fra andre moduler og eksterne afhængigheder som netværksforbindelser og filsystemet.
- Test fejlscenarier, såsom ugyldige URL'er og netværksfejl.
- Brug testframeworks såsom **JUnit (Java), Mockito (Java), xUnit (C#), Moq (.Net) PyUnit(Python), Mock (Python)**, eller lignende afhængigt af sproget

Del 2: Integrationstest

- Hvis man ikke er familiær med integrationstest, kan man gå på opdagelse i nedenstående kilder:

    - https://en.wikipedia.org/wiki/Integration_testing
    - https://martinfowler.com/articles/practical-test-pyramid.html
- Test interaktionen mellem PDF-downloaderen og Excel/CSV-data.
- Sikre, at programmet korrekt læser og behandler URL'er fra inputfilen.
- Verificér, at downloadede filer bliver korrekt registreret med status i outputfilen.
- Undgå brug af mocks – testene skal interagere med faktiske filer

Del 3: Dokumentation

- Evaluér kodekvalitet, herunder:
    - Vurder fx om koden følger ensartede navngivningskonventioner, og at klasser og metoder er logisk organiseret.
    - Vurder om koden er let at forstå og vedligeholde. Tjek om relevante kommentarer er tilføjet, hvor det er vigtigt.
    - Vurder om fejl håndteres korrekt med passende fejlmeddelelser og logging, så problemer kan identificeres og rettes effektivt.
- Evaluér testdækning og potentielle forbedringer.
- Kommentér i din rapport på kodeområder hvor testene har afsløret fejl eller ineffektiviteter.

Del 4 (Valgfri): CI/CD-pipeline

- Hvis du når at færdiggøre testene, kan du implementere en simpel CI/CD-pipeline.
- Brug GitHub Actions, Jenkins eller et lignende værktøj til at køre testene automatisk ved hver commit.
- Overvej at følge et Test-Driven-Development (TDD) arbejdsflow ved retning af bugs- eller forbedring af kodekvaliteten.

Pensum og Ressourcer

- Testning